

Cryptography Algorithms in Python

Caesar Cipher

```
def caesar_cipher(text, shift, mode='encrypt'):
    result = ""
    if mode == 'decrypt':
        shift = -shift

    for char in text:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - base + shift) % 26 + base)
        else:
            result += char

    return result
```

Playfair Cipher

```
def playfair_cipher(text, key, mode='encrypt'):

    def generate_matrix(key):
        key = key.upper().replace("J", "I")
        matrix = []
        used = set()
        for char in key + "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
            if char not in used:
                used.add(char)
                matrix.append(char)
        return [matrix[i:i+5] for i in range(0, 25, 5)]

    def find_pos(matrix, char):
        for i in range(5):
            for j in range(5):
                if matrix[i][j] == char:
                    return i, j

    def process_text(text):
        text = text.upper().replace("J", "I").replace(" ", "")
        pairs = []
        i = 0
        while i < len(text):
            a = text[i]
```

```

        b = text[i+1] if i+1 < len(text) else 'X'
        if a == b:
            pairs.append((a, 'X'))
            i += 1
        else:
            pairs.append((a, b))
            i += 2
    return pairs
matrix = generate_matrix(key)
pairs = process_text(text)
result = ""
for a, b in pairs:
    r1, c1 = find_pos(matrix, a)
    r2, c2 = find_pos(matrix, b)
    if r1 == r2:
        if mode == 'encrypt':
            result += matrix[r1][(c1 + 1) % 5] +
matrix[r2][(c2 + 1) % 5]
        else:
            result += matrix[r1][(c1 - 1) % 5] +
matrix[r2][(c2 - 1) % 5]
    elif c1 == c2:
        if mode == 'encrypt':
            result += matrix[(r1 + 1) % 5][c1] + matrix[(r2 +
1) % 5][c2]
        else:
            result += matrix[(r1 - 1) % 5][c1] + matrix[(r2 -
1) % 5][c2]
    else:
        result += matrix[r1][c2] + matrix[r2][c1]
return result

# Example
print(playfair_cipher("HELLO", "KEY"))

```

Vigenere Cipher

```

def vigenere_cipher(text, key, mode='encrypt'):
    result = ""
    key = key.lower()
    key_index = 0

    for char in text:
        if char.isalpha():

```

```

        shift = ord(key[key_index % len(key)]) - ord('a')
        if mode == 'decrypt':
            shift = -shift
        base = ord('A') if char.isupper() else ord('a')
        result += chr((ord(char) - base + shift) % 26 + base)
        key_index += 1
    else:
        result += char

return result

```

DES (PyCryptodome)

```

from Crypto.Cipher import DES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad

key = get_random_bytes(8) # DES key must be 8 bytes
cipher = DES.new(key, DES.MODE_ECB)
data = b"Hello DES"
ciphertext = cipher.encrypt(pad(data, DES.block_size))

# Decrypt
decipher = DES.new(key, DES.MODE_ECB)
plaintext = unpad(decipher.decrypt(ciphertext), DES.block_size)

print("Ciphertext:", ciphertext)
print("Decrypted:", plaintext)

```

AES (PyCryptodome)

```

from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad

key = get_random_bytes(16) # 128-bit key
cipher = AES.new(key, AES.MODE_ECB)
data = b"Hello AES Encryption"
ciphertext = cipher.encrypt(pad(data, AES.block_size))

# Decrypt
decipher = AES.new(key, AES.MODE_ECB)
plaintext = unpad(decipher.decrypt(ciphertext), AES.block_size)

print("Ciphertext:", ciphertext)
print("Decrypted:", plaintext)

```